

OUTLIERS

agent-flow: Watch Your Agents Hand Work to Each Other, Live

A free screen that draws every Claude Code session, and every second Claude a session starts to do a job, while the work happens. Installed with our fixes, so it cannot leave your Claude Code settings wiped.

PIECE 1 OF 4 • THE AGENT WORKSPACE

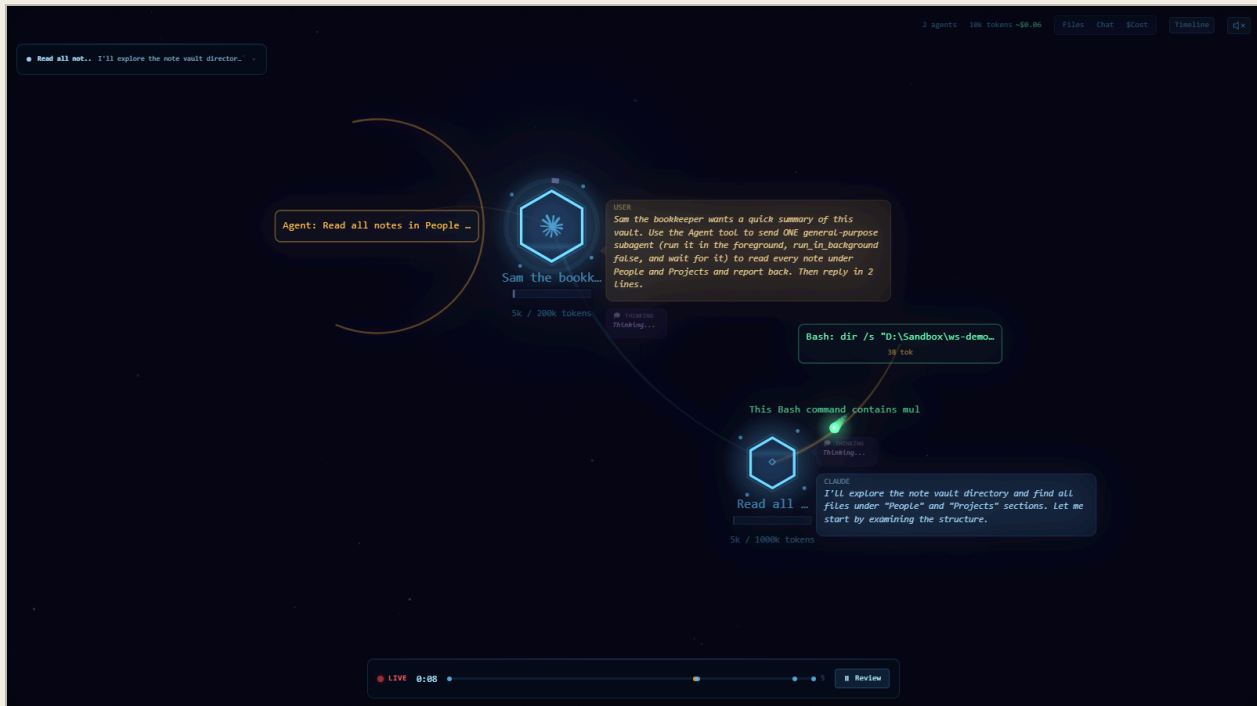
DOWNLOAD

<https://github.com/OUTLIERS-ai/outliers-ws-01-agent-flow>

EST. MMXXVI • OUTLIERS ACCELERATOR

What it is

agent-flow is a free web page that runs on your own computer. Open it in your browser at <http://127.0.0.1:3001> (an address that only your own computer can open; 3001 is the number the page answers on) while Claude Code is working, and you see a dark screen with a glowing hexagon. That hexagon is your Claude Code session. When the session hands a job to a subagent (a second Claude that does 1 task and reports back), a second hexagon appears and a line is drawn between them. Every tool call, such as reading a file or running a command, pops up as a small card beside the agent that made it.



A CLAUDE CODE SESSION (TOP) HAS JUST HANDED A JOB TO A SUBAGENT (BOTTOM RIGHT). MADE-UP DEMO VAULT, TAKEN 2026-09-22: THE NAMES IN IT, SUCH AS SAM THE BOOKKEEPER, ARE INVENTED.

Along the top of the page are tabs, 1 per Claude Code session running on your computer. Each tab is titled with the first words of that session's opening message. Top right shows how many agents are running, an estimate of tokens used (tokens are the units Claude's usage is counted and billed in) and an estimated cost. Along the bottom is a timeline with a LIVE marker and a Review button that replays what happened. There is also a sound button in the top right corner, which mutes the page's small sound effects.

agent-flow was written by another developer and published free, with its code open for anyone to read and change (the Apache 2.0 licence; code at github.com/patoles/agent-flow). It is listed as `agent-flow-app` on npm, the public store of Node.js programs, version 0.9.1 checked on 2026-09-22. It also comes as an add-on for the VS Code code editor. Our download does not contain agent-flow. It contains 5 small Python scripts: an installer (`install.py`), a start-and-stop script (`start.py`), 2 checking scripts (`check_hooks.py` and `cleanup.py`) and `guard.py`, a small program that sits between agent-flow and your browser. Together they set agent-flow up safely, because agent-flow installed on its own, unchanged, has faults on Windows that kept coming back from 2026-06-12 to 2026-09-16, and 1 more fault, on Windows and Mac alike, that can throw away your Claude Code settings file and everything in it.

This is piece 1 of 4 in the agent workspace. Install them in order: 1 agent-flow, 2 FleetView, 3 ProjectForge, 4 Jeeves. Each one also works on its own.

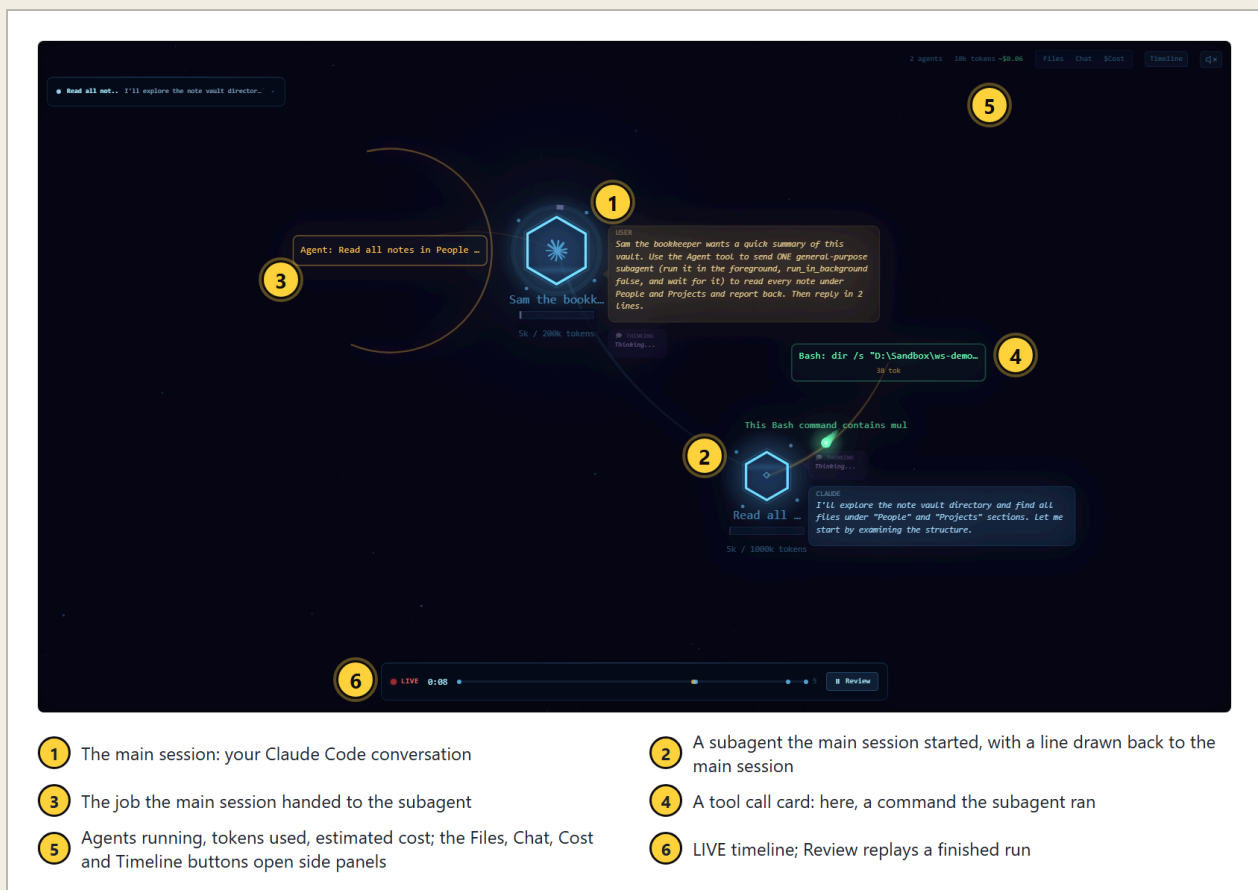
WORDS USED IN THIS GUIDE

WORD	WHAT IT MEANS HERE
Session	1 conversation with Claude Code, running in 1 terminal window.
Subagent	A second Claude that your session starts to do 1 job and report back.
Tool call	1 action Claude takes, such as reading a file or running a command.
Token	The unit Claude's usage is counted in. 4 tokens are roughly 3 words.
Hook	A command Claude Code runs by itself every time a given event happens, such as a tool about to run. Hooks are listed in <code>settings.json</code> .
Terminal	The text window where you type commands: PowerShell on Windows, Terminal on a Mac.
Server	The agent-flow program running in the background on your computer, which draws the page.
127.0.0.1 and localhost	2 ways of writing "this computer". An address that starts with either of them opens only on your own computer.
Port	The number after the colon in an address such as http://127.0.0.1:3001 . It picks out which program on your computer answers. In this set of 4: agent-flow 3001, FleetView 3010, ProjectForge 3020, Jeeves 4040.
Node.js, npm, npx	Node.js runs programs written in JavaScript; npm is its public store of programs; npx downloads a program from npm and runs it.
settings.json	Claude Code's settings file, in the <code>.claude</code> folder inside your home folder.
Repo	A project's folder of code, kept on GitHub. <code>git clone</code> copies it to your computer.
Watch folder	The folder agent-flow listens to; only sessions started inside it appear.
Exit code	The number a program hands back when it finishes; 0 means all went well.

PIECE 1 • AGENT-FLOW

Why you would want it

When you run agents, most of the work is invisible. You type a request and later an answer arrives. You do not see which subagents were called, in what order, or which files they opened. The 5 situations where that matters are listed under Works well when, below.



WHAT YOU CAN READ OFF THE SCREEN. THE YELLOW NUMBERS ARE OURS, DRAWN ON A MADE-UP DEMO RUN.

agent-flow only reads what your agents do; it cannot change it.

WHAT IT IS FOR

Watching, as it happens, which subagents a Claude Code session starts and which files and commands it uses.

WORKS WELL WHEN

- **You are building an agent.** Its instructions say it calls 3 specialists. You run it with this screen open and see which specialists it really starts, in what order, and every file it really reads.
- **A session is slow or costing more than you expected.** You see the moment it starts 5 subagents at once, or reads the same file 4 times.
- **You are recording your screen** for a video, for a client, or for a live training you are running. Someone watching it understands in 5 seconds what a written description never gets across.
- **You have just changed an agent's instructions** and want to watch the next run rather than read a summary of it afterwards.
- **You want to know why a session has gone quiet:** waiting on 1 slow command, or working through 6 subagents.

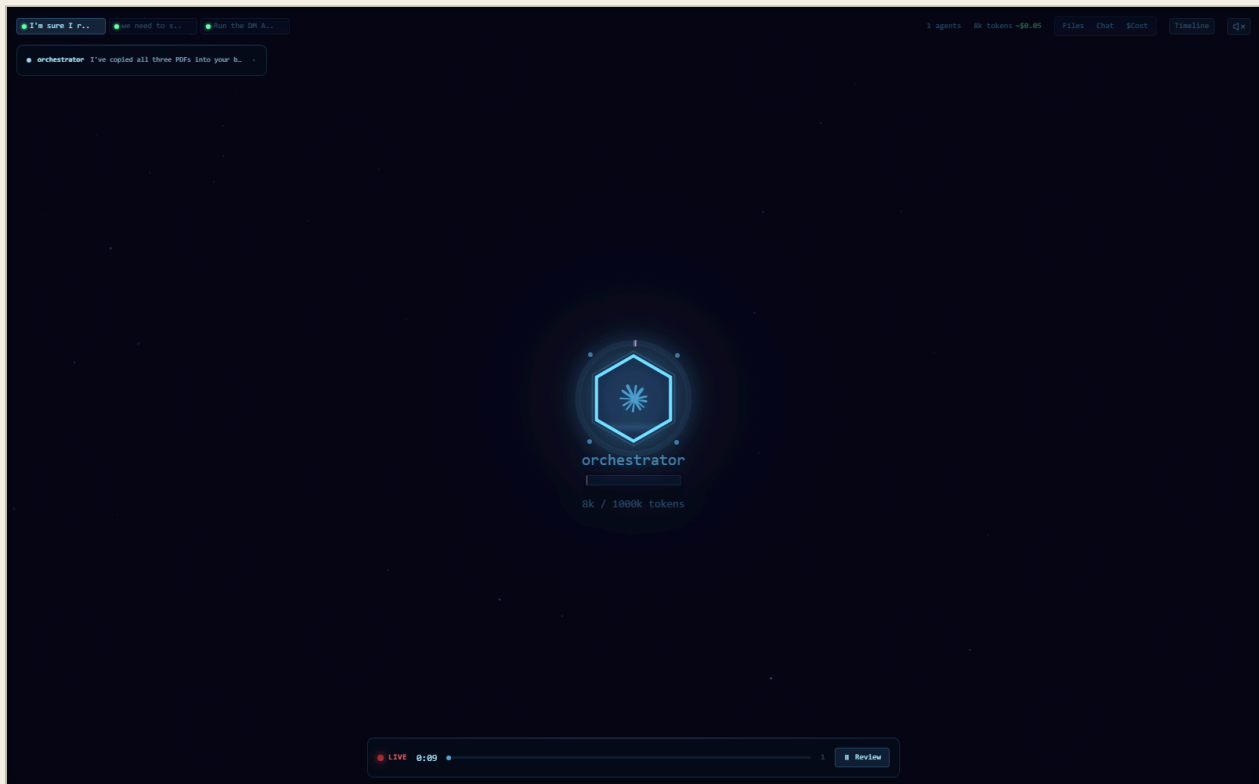
DOES NOT WORK WELL WHEN

- agent-flow keeps nothing. The Review button replays only what the agent-flow program on your own computer is still keeping; stop that program and the record is gone for good. For a written trail of what each agent did, use ProjectForge, piece 3 of 4.
- **You want every session on 1 screen.** It draws 1 session per tab and you click between them. Ashley's attempt to merge them failed on 2026-06-12 (see How we built it). FleetView, piece 2 of 4, is the screen that does this.
- **You are screen-sharing or recording for other people.** Each tab is titled with the opening words of that session's prompt, and the cards beside each agent show your file names and commands. Check the tabs before you share.
- **You want to leave it running all day unwatched.** agent-flow listens for events on a second numbered address of its own, and it accepts made-up events from any website that guesses that number. That is agent-flow's own code, not ours, so the only real fix is a change to agent-flow itself. Stop it when you are not watching: `python start.py --stop`.
- Do not install this piece if you do not want Node.js on your computer. agent-flow needs Node.js, and once agent-flow's small command has been added to Claude Code's settings file, every tool call starts Node.js once, even when the screen is off, until you uninstall.

PIECE 1 • AGENT-FLOW

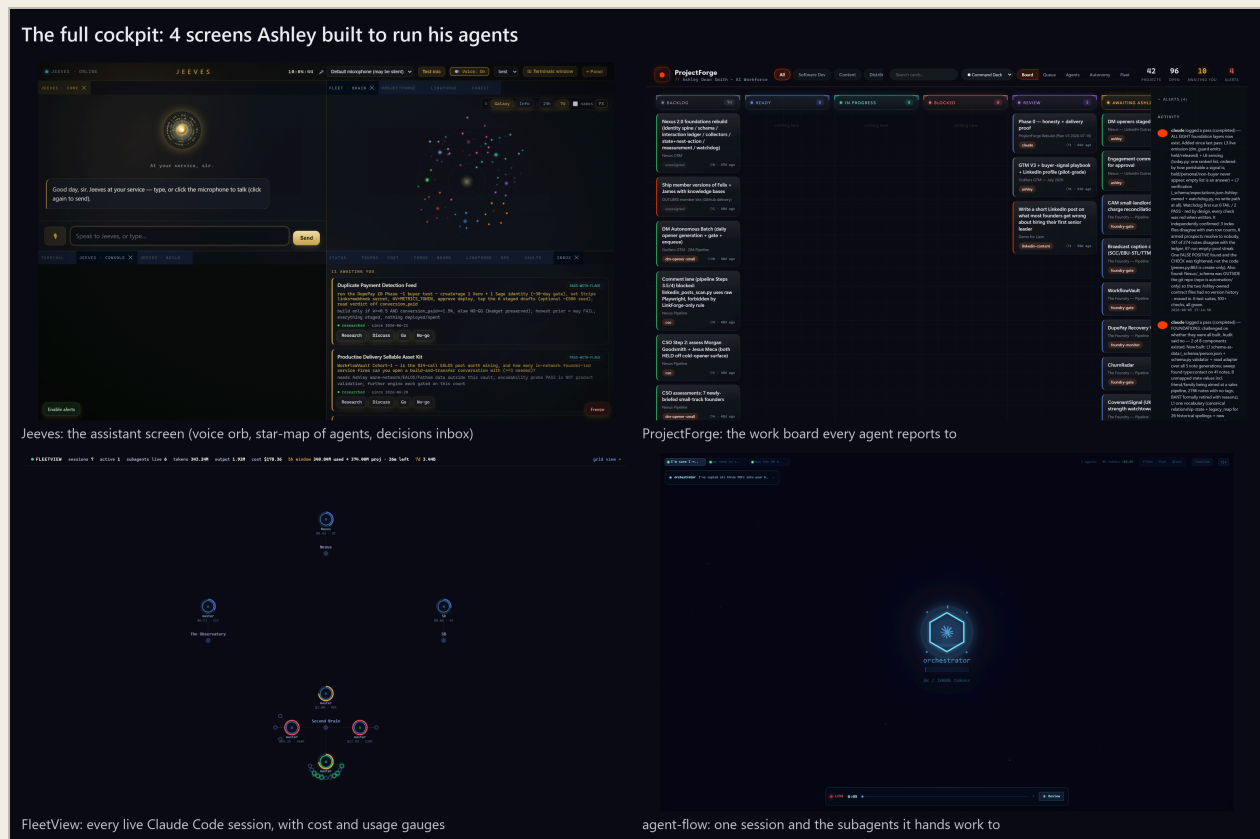
How we built it

This is how Ashley set agent-flow up on his own Windows PC, what broke, and what we kept. Every date and count below was read from his files, settings backups and logs on 2026-09-22.



ASHLEY'S OWN AGENT-FLOW, RUN ONCE WITH NO WINDOW ON 2026-09-22 FOR THIS GUIDE. THE 3 TABS ALONG THE TOP ARE HIS REAL CLAUDE CODE SESSIONS, TITLED WITH THE FIRST WORDS OF EACH PROMPT.

agent-flow is 1 of 4 screens Ashley built around his agents. The other 3 each have their own guide in this set of 4: FleetView, ProjectForge and Jeeves.

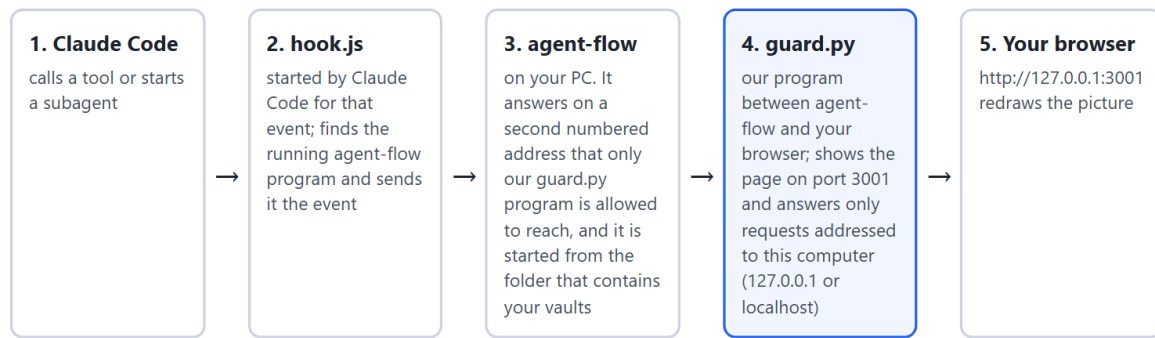


ASHLEY'S REAL SCREENS SIDE BY SIDE, 2026-09-22: JEEVES (A PERSONAL-ASSISTANT SCREEN OVER HIS SECOND BRAIN), PROJECTFORGE (A BOARD OF WORK CARDS HIS AGENTS PICK UP), FLEETVIEW (EVERY RUNNING CLAUDE CODE SESSION AND ITS COST ON 1 PAGE) AND AGENT-FLOW (BOTTOM RIGHT).

Claude Code lets you register a **hook**: a command it runs by itself every time a given event happens, such as "a tool is about to be used" or "a subagent has started". agent-flow registers a small script, `hook.js`, on 9 kinds of event. Each time 1 of them happens, Claude Code runs `hook.js` and hands it a description of the event. The script looks in the folder `<your home>/.claude/agent-flow/` for small files that say where the agent-flow server (the agent-flow program running in the background, which draws the page) is listening, and sends the event there. The server updates its picture and your browser redraws.

The hook is only half of what draws the page: the server also reads Claude Code's own session transcript files, in `<your home>/.claude/projects/`, and that is where the tab title, the conversation in the Chat panel, the token counts and every subagent come from. The second hexagon, and the line joining it to the first, are drawn from the place in that transcript where the session starts the subagent, not from the "a subagent has started" event, which records only a name.

How 1 event travels from Claude Code to your screen



Steps 1 to 3 are agent-flow's own design. Step 4 is ours, added on 2026-09-22: without it, a website could read your conversation as it happens. Nothing on this path leaves your computer.

HOW 1 EVENT TRAVELS. STEP 4, GUARD.PY (OUR PROGRAM BETWEEN AGENT-FLOW AND YOUR BROWSER), IS OUR ADDITION FROM 2026-09-22: IT REFUSES REQUESTS FROM OTHER WEBSITES. THE SECURITY SECTION BELOW EXPLAINS WHY.

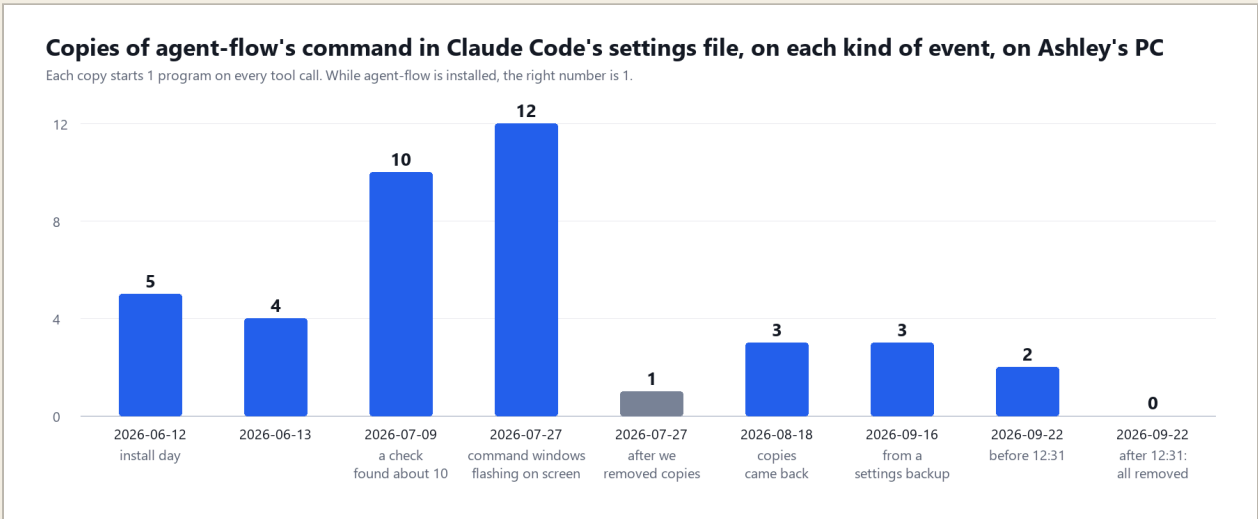
2026-06-12: INSTALL DAY

- Installed with `npx -y agent-flow-app` (the `-y` answers yes to the download question). `npx` comes with Node.js; it downloads a Node.js program and runs it in `1 go`. On its first run the package wrote `hook.js` and added it to 9 kinds of event in Claude Code's `settings.json` file: `SessionStart` (a session opens), `PreToolUse` and `PostToolUse` (before and after each tool call), `PostToolUseFailure` (a tool call fails), `SubagentStart` and `SubagentStop`, `Notification` (Claude Code shows a message), `Stop` (Claude finishes a reply) and `SessionEnd` (a session closes).
- Windows fix 1, the launch folder.** The server remembers the folder it was started from, and only accepts events from Claude Code sessions running inside that folder. Started from the wrong place, it shows nothing. So it was set to start from the Documents folder, which contains every vault.
- Start at logon, with no window.** A small Windows script file, `agent-flow.vbs`, went into the Windows Startup folder (Windows runs everything in that folder when you log in). It runs `npx -y agent-flow-app --no-open`, and `--no-open` stops it opening a browser tab.
- Every session on 1 screen, tried and undone.** Ashley wanted every session on 1 screen instead of 1 tab each. We changed agent-flow's own code so it sent every session to the same screen. The data looked right, and it was reported as done without anyone looking at the screen. Ashley's reply: "that hasn't worked - It's not working at all." agent-flow's drawing code supports exactly 1 main hexagon per screen, so 6 main hexagons stacked on the same spot and their labels turned to nonsense. agent-flow was put back exactly as its author published it. Since then, every change to a screen is checked with a screenshot, never by reading the data behind it.
- The 1-screen wish went to a different tool, `FleetView`, which is piece 2 of 4.
- A backup of `settings.json` taken at 17:00 that day already had **5 copies** of the agent-flow hook on every event. The next day's backup had 4.

WINDOWS FIX 2: OLD FILES POINTING AT SERVERS THAT HAD STOPPED (DATE NOT RECORDED)

Each time the server starts it writes a small file into `<home>/ .claude/agent-flow/` . Its name is a random code, a dash, the process number (the number Windows gives each running program), then `.json` , and inside it says which port that server listens on and which folder it watches. This guide calls those files **registration files**. The hook is meant to delete files for servers that have stopped. On Windows its "is this server still alive?" check is written to always answer yes. Worse, when several files match a session's folder, the hook sends only to the server watching the **smallest** folder that contains the session (Documents\Second Brain is smaller than Documents, because it sits inside it). So if an old file names a server for a smaller folder, and that server has stopped, every event goes to it and is lost, the real server gets nothing, and the screen stays empty. The fix was to delete the old files and keep the file for the server that is running. 13 such files were still in that folder on 2026-09-22, dated 2026-06-12 to 2026-08-06.

2026-07-09 TO 2026-09-16: THE HOOK COPIES PILE UP



COPIES OF THE AGENT-FLOW HOOK ON EACH KIND OF EVENT, FROM SETTINGS BACKUPS AND CHECKS OF THE SETTINGS FILE.

- **2026-07-09:** a check of the settings file found the hook registered about 10 times on every event.
- **2026-07-19:** `npx` downloaded the newer agent-flow, version 0.9.1.
- **2026-07-27:** Ashley: "Instances keep opening on my PC." The settings file had 112 hook entries in total; agent-flow's was on every event 12 times. Each tool call started about 27 small command-window programs, and each flashed a window on screen. Every hook was cut to 1 copy per event and command: 112 entries down to 13.
- **2026-08-18:** the copies had come back: 3 per event.
- **2026-09-16:** a backup shows 3 per event. On 2026-09-22 the file had 2 copies per event, and all of them were removed that day. No written record was found of the drop from 3 to 2.

At the time, why the copies piled up was not recorded. While building this kit on 2026-09-22 we read agent-flow 0.9.1's own set-up code and found a cause. Before adding its hook, agent-flow checks whether it is already there by searching your settings for the text `agent-flow/hook.js` , with a forward slash. On Windows the path it writes uses backslashes: `agent-flow\hook.js` . The search never matches its own entry, so **every time the server starts, it adds another copy to all 9 kinds of event**. We tested it in a

test folder: 3 starts gave 3 copies per event. A server that starts at every logon, plus starts by hand during fixes, matches every rise in the chart. We cannot prove it was the only cause, because nobody logged each start.

Our fix changes only the direction of the slashes: the installer writes the same hook with forward slashes (`C:/Users/<you>/.claude/agent-flow/hook.js`). Windows accepts forward slashes, and agent-flow now recognises its own entry and leaves your settings alone. Tested the same day: with our forward-slash version of the hook line in settings.json, starting agent-flow twice more added no further copies.

THE USAGE-TRACKING FINDING

agent-flow 0.9.1 sends anonymous usage events (an install code, its version, your operating system, session length) to a server run by its author, unless 1 of 2 switches is set before it starts:

`AGENT_FLOW_TELEMETRY=false` or `DO_NOT_TRACK=1` . These are environment variables: settings a program reads from Windows or macOS when it starts. The GitHub page summary says tracking is off by default; the code switches it on. On Ashley's PC, 4 such events were logged between 2026-07-19 and 2026-08-06, because the file that started his agent-flow at logon set neither switch. Ours sets both.

2026-08-06: SWITCHED OFF

At 17:25 on 2026-08-06 the Startup entry for agent-flow was switched off in Windows' Startup apps list, in the same minute as the FleetView dashboard. Who did it and why was not written down. A note Ashley wrote on 2026-09-22 says these screens were "stopped not deleted" when his work changed direction. The hooks were left in place, so every tool call still started Node.js (the program that runs agent-flow's scripts) once, found no server and quit. Our uninstall removes those hooks.

2026-09-22: WHAT A SECURITY CHECK OF OUR OWN KIT FOUND

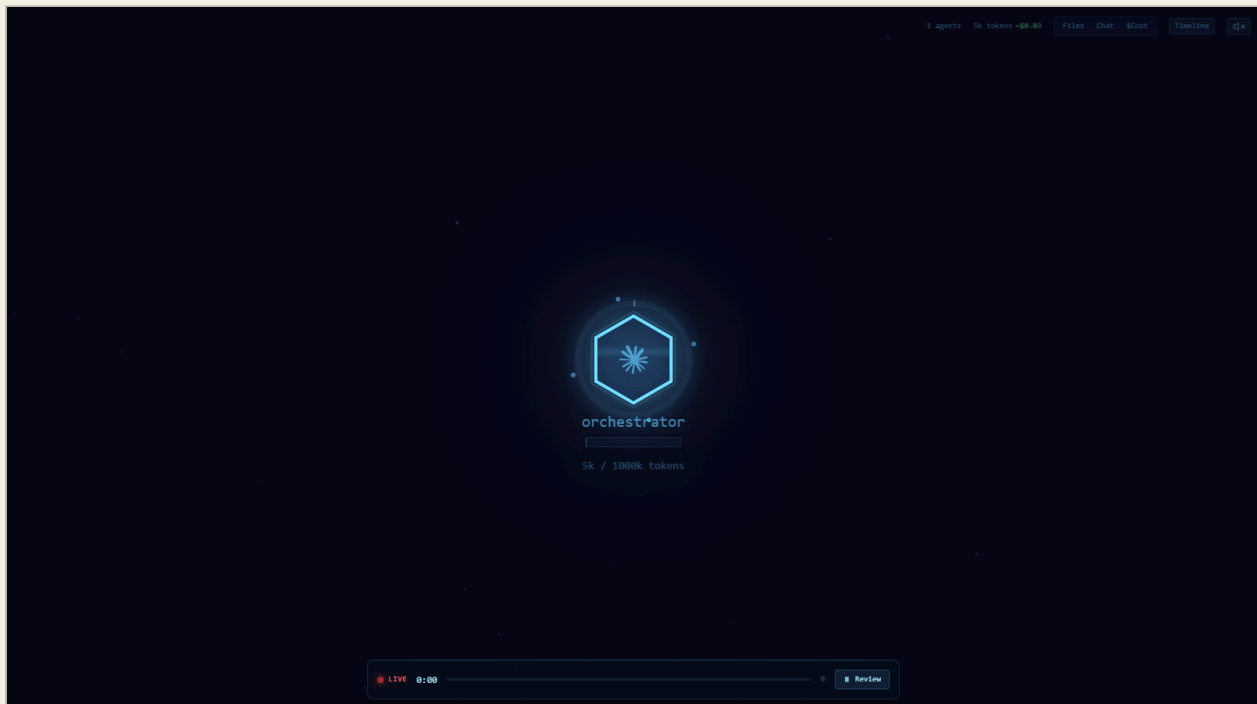
Before this guide went out, our 5 Python scripts (the kit you download) were tested by someone trying to break them, using the real agent-flow in a test home folder. 3 faults were found and all 3 are fixed in the version you download.

- **agent-flow could wipe your Claude Code settings at logon.** agent-flow runs its own set-up every time it starts. If it cannot read `settings.json` , it throws the whole file away and writes a new one containing only its 9 hooks: your permission rules, your blocked commands, your model choice and every other hook, gone, with no backup. 2 ordinary slips make the file unreadable to it: a comma after the last item (easy to leave after a hand edit), and an invisible marker at the very start of the file called a byte-order mark, which Windows PowerShell and some editors add when they save. With agent-flow starting by itself at logon, that would happen the next time you logged in, with nothing on screen to tell you. **Our fix:** `start.py` now checks the file before every start and does not start agent-flow if the file is unsafe; it says why on screen and in `logs/start.log` . There is also a second check: a new file, `guard.py` , keeps an exact copy of your settings before agent-flow starts, watches the file for the first 60 seconds, and puts your copy back if agent-flow changed it. Tested with the real agent-flow: with a comma after the last item, or with a byte-order mark, your settings file is left exactly as it was, character for character.
- **Stop could close an unrelated program.** `start.py --stop` used to stop whatever program had the process number saved when agent-flow last started. After a restart, Windows can give that number to any program, including Claude Code or your browser. **Our fix:** the saved record now includes the moment the program started, and stop only acts when both match.

- **Other websites could read the page.** agent-flow's page shows your conversation as it happens: your prompts, Claude's replies, file names and commands. It answers every request that reaches it, and never checks which website the request came from. A website you visit can trick your browser into sending its requests to your own computer, and then read your conversation while you browse. **Our fix:** agent-flow now answers on a second port that only `guard.py` talks to, and `guard.py` shows you the page at <http://127.0.0.1:3001>. It answers only requests addressed to this computer (`127.0.0.1`, `localhost` and `[::1]` are 3 ways of writing "this computer"). Anything else is refused.

WARNING

1 weakness remains. It is in agent-flow's own code, so we cannot fix it from outside. The separate port that receives events from `hook.js` accepts them from any web page, so a website that guesses that port number (it changes at every start) could draw fake sessions and fake tool calls on your screen. It cannot read anything through that port. If you are not watching agent-flow, stop it with `python start.py --stop`. A real fix needs a change in agent-flow itself (github.com/patoles/agent-flow).



THE PAGE AS `GUARD.PY` (OUR PROGRAM BETWEEN `AGENT-FLOW` AND YOUR BROWSER) PASSES IT TO YOUR BROWSER, FROM A MADE-UP SESSION ON 2026-09-22: 1 AGENT, AND 5,000 TOKENS USED OF THE 1,000,000 THIS SESSION MAY USE. "ORCHESTRATOR" IS THE NAME `AGENT-FLOW` GIVES THE MAIN SESSION.

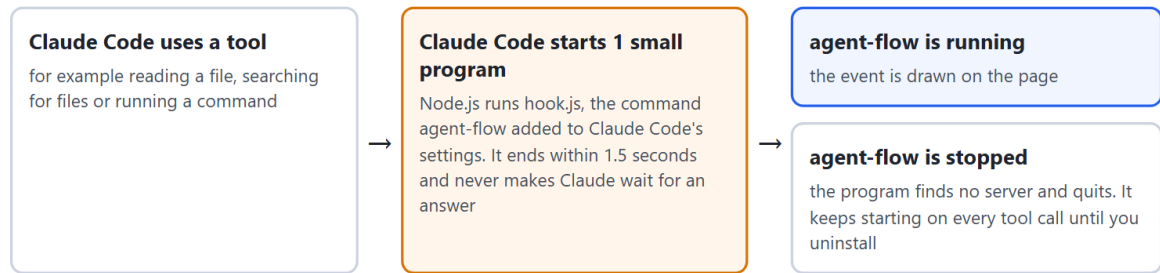
WHAT WE KEPT FOR YOU

- agent-flow exactly as its author published it, downloaded by your own computer from npm. We changed none of its code.
- The hook entry in settings.json written with forward slashes (/), so agent-flow recognises it and never adds another copy.
- A small file in your Startup folder that starts agent-flow when you log in, with no window, from the folder that contains your vaults, with usage tracking off.
- `cleanup.py`, which deletes the registration files a stopped agent-flow server left behind saying where it was listening, and ends a second server left watching the same folder, run automatically before every start.
- `check_hooks.py`, a count of every hook on every event.
- `guard.py`, which puts settings.json back if agent-flow wipes it and refuses page requests from other websites (both found when we tried to break the kit on 2026-09-22).
- A start that fails now tells you why in 1 sentence, instead of pointing you at a log full of Node.js text (added 2026-09-23: we started agent-flow 34 times in a row, 1 start failed, and the message it printed could not be read).

Pros and cons

	PROS	CONS
Cost	Free, with its code open to read. Uses no Claude tokens itself.	Every tool call starts Node.js once, to run <code>hook.js</code> (it ends within 1.5 seconds and never makes Claude wait). It does this even when the server is off, until you uninstall.
Setup	About 1 minute with our installer, plus the first download.	Needs Node.js, which most people do not have yet.
What you see	A subagent appears within a second of starting, read from Claude Code's own session transcript file; every tool call is a card beside the agent that made it; the Review button at the bottom of the page replays a finished run.	1 session per tab. There is no single screen with every session: our 2026-06-12 attempt to add that screen failed. FleetView (piece 2 of 4) is that screen.
Accuracy	Tool calls come straight from Claude Code's own events; subagents, the conversation and the token counts are read from Claude Code's session transcript files.	Token counts and costs are estimates. On Ashley's PC they disagreed with FleetView (the session-and-cost screen in piece 2 of 4) for the same session.
Privacy	The page only listens on your own computer, and guard.py (our program between agent-flow and your browser) refuses requests addressed to anything but your own computer.	Tab titles show the first words of your prompts, so take care when screen-sharing. Usage tracking is on unless switched off (our start-at-logon file switches it off). A website that guesses a port number can still draw fake sessions on your screen.
Safety	Our kit will not start agent-flow on a settings file it would wipe, and puts the file back if it changes anyway.	agent-flow still runs its own set-up every time it starts. Our checks run before and after that set-up; they cannot stop it happening.
Windows	Our kit fixes the 2 known Windows faults: the start folder, and leftover registration files that swallow events. Closing agent-flow by force (from Task Manager, or a crash) used to leave a second server running and sending the same events twice; <code>cleanup.py</code> now counts how many agent-flow servers are running, instead of how many folders are being watched, ends the extra server, and prints a line saying it did.	agent-flow's own registration files are still written by agent-flow, so a server closed by force leaves its file behind until the next <code>python start.py</code> or <code>python cleanup.py</code> .

What happens on every tool call, while agent-flow is installed



9 kinds of Claude Code event carry agent-flow's command: a session opens, a session closes, before a tool runs, after a tool runs, a tool fails, a subagent starts, a subagent stops, Claude shows a message, and Claude finishes a reply. agent-flow uses no Claude tokens.

THE COST OF HAVING THE HOOK INSTALLED: 1 SHORT-LIVED PROGRAM PER TOOL CALL, WHETHER OR NOT THE PAGE IS OPEN.

PIECE 1 • AGENT-FLOW

Before you start

YOU NEED	HOW TO CHECK	IF IT IS MISSING
Claude Code	<code>claude --version</code>	You have it from earlier Outliers Accelerator sessions.
Python 3.11 or newer	<code>python --version</code> (Mac: <code>python3 --version</code>)	python.org. <code>install.py</code> tests this and stops if it is older, because Python 3.8 stopped getting security fixes on 2024-10-07 and 3.10 stops on 2026-10-31.
Git	<code>git --version</code>	git-scm.com
Node.js 22 or newer	<code>node --version</code>	Get whichever version nodejs.org offers as LTS on the day (LTS means long-term support, the version that goes on getting security fixes longest); the test PC in the picture below ran 26.3.0. Node.js 18 stopped getting security fixes on 2025-04-30 and Node.js 20 on 2026-04-30, so <code>install.py</code> stops on anything below 22. Windows: the LTS installer from nodejs.org, or <code>winget install OpenJS.NodeJS.LTS</code> (winget is Windows' built-in install command). Mac: nodejs.org or <code>brew install node</code> (if you have Homebrew, a Mac install tool). Then open a NEW terminal (the text window where you type commands), because a terminal opened before the install cannot find Node.js.
Port 3001 free (the number the page answers on)	Open http://127.0.0.1:3001 in a browser; it should fail to load	Install with <code>python install.py --port 3002</code> instead, then use 3002 wherever this guide says 3001.

Windows PowerShell - the 4 checks, as they printed on a test PC on 2026-09-22

```
> claude --version
2.1.278 (Claude Code)
> python --version
Python 3.13.14
> git --version
git version 2.54.0.windows.1
> node --version
v26.3.0
```

THE 4 CHECKS AS THEY PRINTED ON A TEST PC ON 2026-09-22. YOUR VERSION NUMBERS CAN BE HIGHER.

You also need to know where your second brain vault and your CRM vault live. The installer finds them for you in 1 of these ways: small files called `.outliers-sb` and `.outliers-crm` in your home folder, each containing the path to 1 of your vaults (the installers from earlier Outliers Accelerator sessions left them there), a folder with an `.obsidian` folder inside it (that is what makes a folder an Obsidian vault) up to 2 levels under your home or Documents folder, or, for the CRM, a vault whose name contains `crm`, `pipeline`, `sales` or `clients`, or a folder at `C:\Users\<you>\CRM`.

PIECE 1 • AGENT-FLOW

Install it

1. Open a terminal. Windows: press the Windows key, type `PowerShell`, press Enter. Mac: open Terminal. It opens in your home folder, which is where all 4 downloads in this set go.
2. Run the clone and install line:

```
git clone https://github.com/OUTLIERS-ai/outliers-ws-01-agent-flow; cd outliers-ws-01-agent-flow; python install.py
```

On a Mac use `python3`. Keep the folder where it lands: the small file that starts agent-flow when you log in points at it.

3. Answer 3 questions. Press Enter to accept the suggestion in square brackets.
 - Where is your second brain vault?
 - Where is your CRM vault?
 - Which folder should agent-flow watch? It suggests the folder that contains both vaults. It must contain every folder you run Claude Code in, or those sessions will not show.

If a question shows no suggestion, paste the full path of that vault (in File Explorer, click the address bar and copy it). No CRM vault yet? Press Enter to leave it blank.

4. Wait for the first run. The installer downloads agent-flow and starts it once, with no window, pointed at a temporary empty folder that stands in for your home folder, so it writes its own `hook.js` without touching your settings. It copies that script into your `.claude/agent-flow` folder and stops it. This can take up to 3 minutes on a slow connection. No login is needed.

- Read the table the installer prints at the end: 1 row per kind of event. Every event should show 1 in the "after" column. If you had old copies, the "before" column shows how many, and a dated backup of your settings is named on screen. The file that starts agent-flow at logon is `outliers-agent-flow.vbs` in your Startup folder; that is the name you see in Windows' Startup apps list.

What a first install prints (3 times Enter pressed). `install.py` checks your Python and Node.js versions before anything else.

```
> python install.py
agent-flow installer (Outliers Accelerator, guide 1 of 4)

Python 3.13.14 found.
Node.js 26.3.0 found.
Where is your second brain vault? [C:\Users\<you>\Documents\Second Brain]:
Where is your CRM vault? [C:\Users\<you>\Documents\Demo CRM]:
Which folder should agent-flow watch? It must CONTAIN every vault you run Claude Code in [C:\Users\<you>\Documents]:
Saved settings to C:\Users\<you>\outliers-ws-01-agent-flow\config.json
First run: downloading agent-flow-app@0.9.1 and letting it write its hook script (can take up to 180 seconds)...
Hook script present: C:\Users\<you>\.claude\agent-flow\hook.js

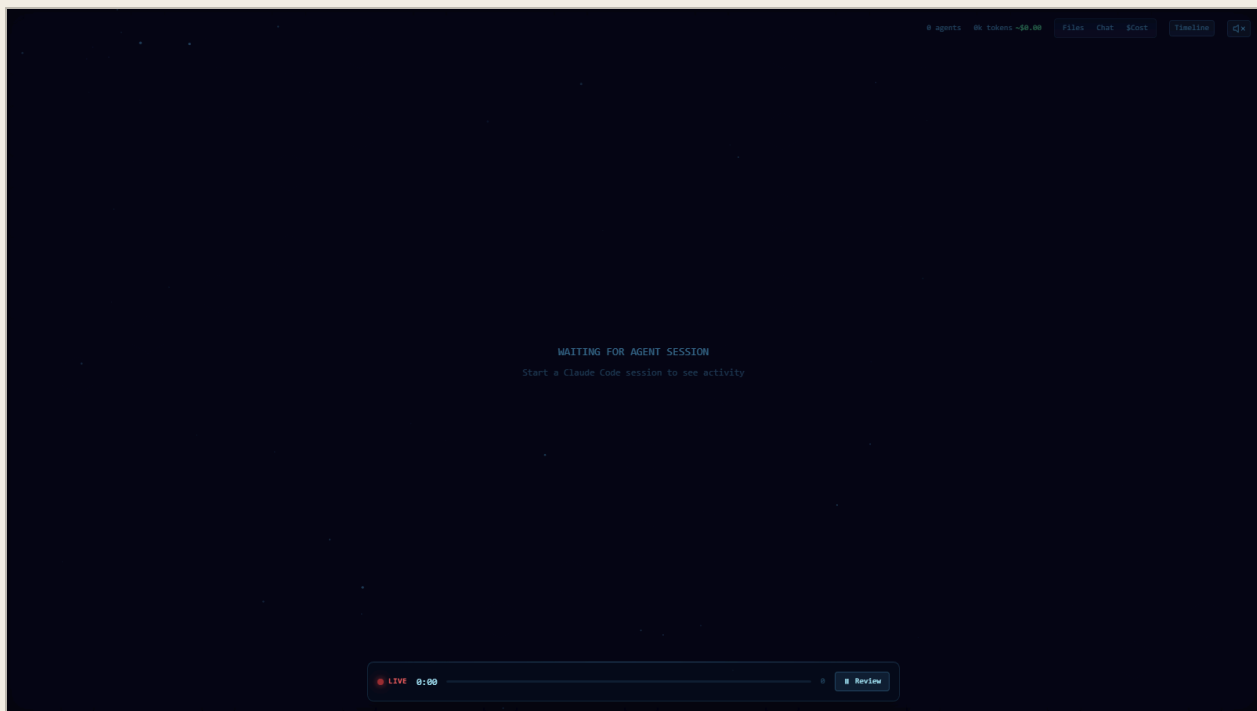
Claude Code settings: C:\Users\<you>\.claude\settings.json
Event          before  after
SessionStart   0       1
PreToolUse     0       1
PostToolUse    0       1
PostToolUseFailure 0       1
SubagentStart  0       1
SubagentStop   0       1
Notification   0       1
Stop           0       1
SessionEnd     0       1
Backup taken first: C:\Users\<you>\.claude\settings.json.bak-agent-flow-20260923-000109
Removed 0 extra copies; each event now has exactly 1.
The hook line now uses forward slashes, so agent-flow recognises it and will not add another copy each time it starts.

File that starts agent-flow at logon, written: C:\Users\<you>\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\outliers-agent-flow.vbs
Usage tracking: off (AGENT_FLOW_TELEMETRY=false and DO_NOT_TRACK=1 are set by that file).

Done. Start it now with: python start.py
Then open http://127.0.0.1:3001 and start a NEW Claude Code session inside the watch folder.
Check your hooks any time with: python check_hooks.py
```

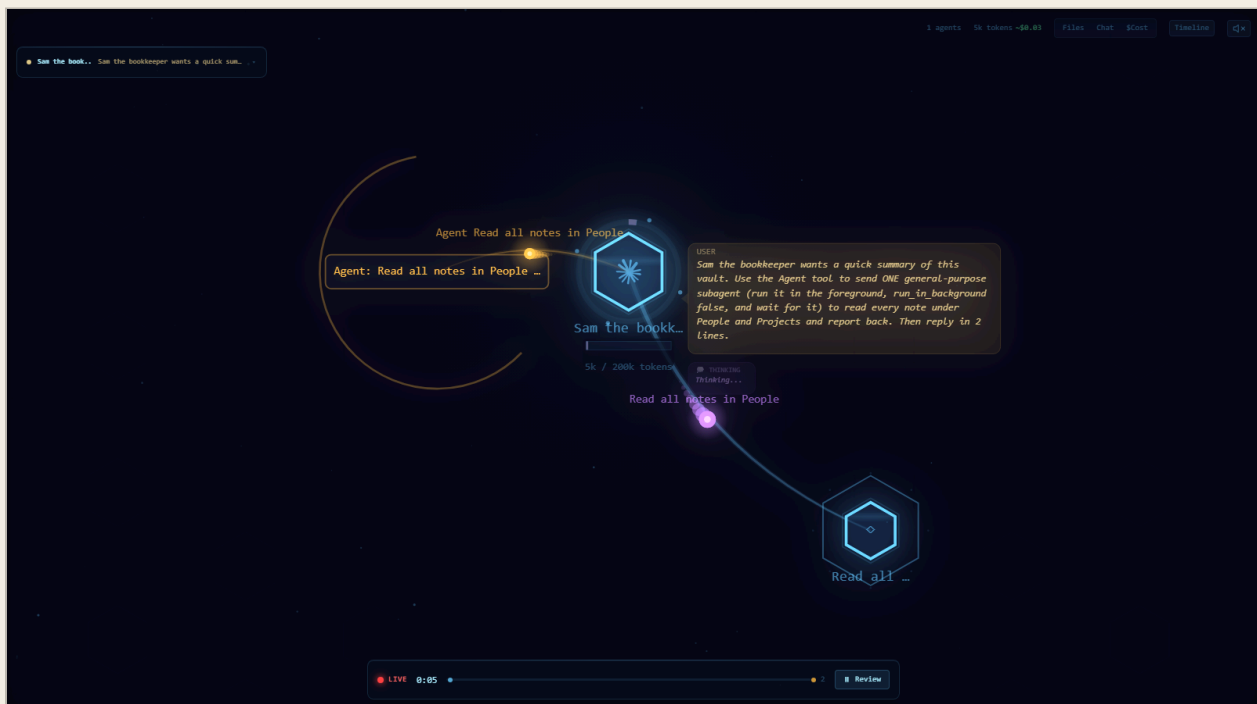
WHAT A SUCCESSFUL INSTALL PRINTS, RETAKEN 2026-09-23 WITH THE 2 VERSION CHECKS IT NOW RUNS FIRST. PATHS SHORTENED TO `C:\USERS\<YOU>`; THE DEMO RAN IN A MADE-UP HOME FOLDER, AND "DEMO CRM" IS AN INVENTED VAULT.

- Start it now: `python start.py`. From now on it also starts by itself, with no window, each time you log in.
- Open <http://127.0.0.1:3001>. In the middle of the page you see "WAITING FOR AGENT SESSION" in capitals, and under it "Start a Claude Code session to see activity".



THE PAGE BEFORE ANY SESSION HAS STARTED.

- Open a **new** Claude Code session inside 1 of your vaults. Sessions that were already open show nothing, because Claude Code reads hooks only when a session starts. Ask it for something that uses a subagent, for example: "Use a subagent to list the notes in my People folder, then summarise them in 2 lines." A hexagon appears, then a second hexagon with a line to it. Nothing after 30 seconds? Check the session was started AFTER step 6 and inside the watch folder shown by `python start.py --status`, then see the "When it goes wrong" table at the end of this guide.



THE MAIN SESSION (THE BRIGHT HEXAGON IN THE MIDDLE) STARTING A SUBAGENT (BOTTOM RIGHT). THE ORANGE LABEL ON THE LEFT IS THE JOB THE SUBAGENT WAS GIVEN.

- Run `python check_hooks.py`. It should end with `RESULT: OK`.

TIP

to remove everything later, run `python install.py --uninstall`. It stops the server, takes a backup, removes the 9 hook entries, and deletes the file that starts agent-flow at logon. Your settings go back to exactly how they were, character for character, as long as the backup from your first install is still there. If you had no settings file before, an empty settings file (containing only `{}`) is left. The folder `.claude\agent-flow` stays; you may delete it by hand.

PIECE 1 • AGENT-FLOW

Using it day to day

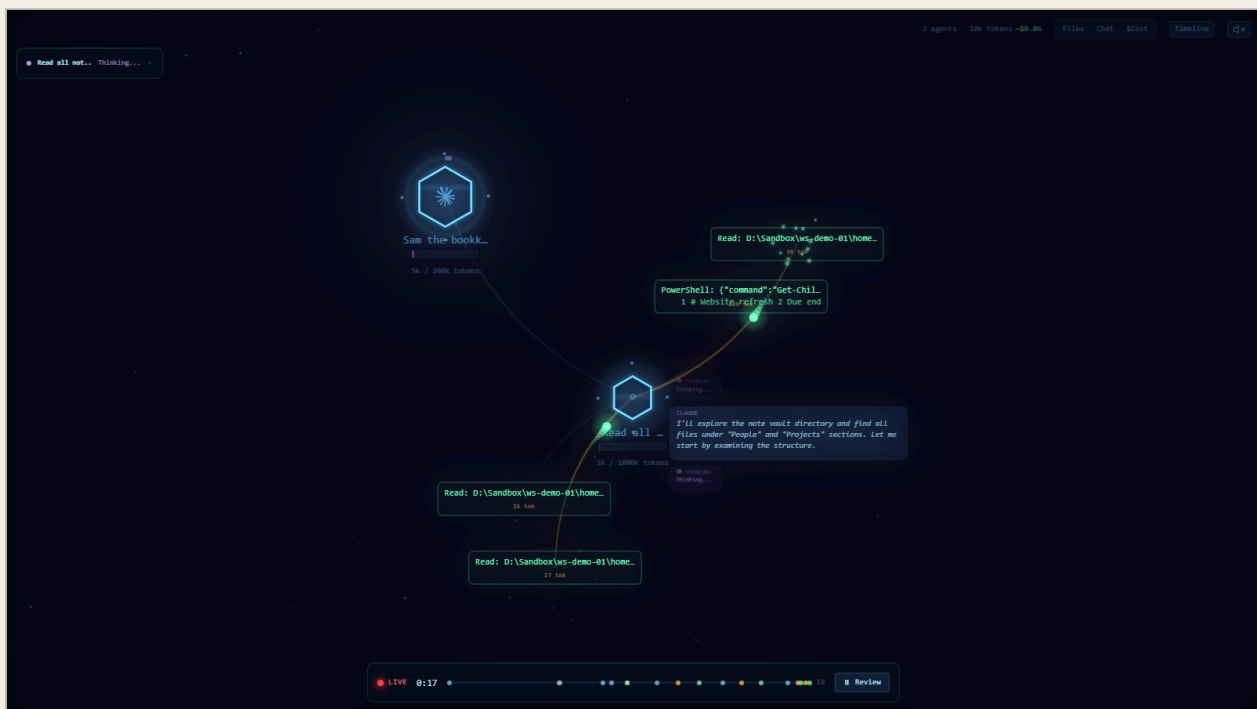
Every command below runs inside the downloaded folder. In a new terminal, type `cd outliers-ws-01-agent-flow` first.

- **Leave the page open in a browser tab** while you work. Switch tabs at the top to follow a different session.
- **Press Review** on the bottom bar to replay a run that has finished.
- **The Files, Chat and Timeline buttons** top right open side panels: which files were touched, the conversation, and a timeline of every call.
- **The \$Cost button** beside them opens no panel. It writes an estimated price above each hexagon and draws a small cost box in the corner of the picture, split by agent and by tool. Nothing appears until a session has used enough tokens for the estimate to come to more than \$0.00.
- **Check your hooks once a week:** `python check_hooks.py`. Anything other than 1 per event means something else has been writing to your settings.
- **If the page goes quiet:** run `python start.py --stop`, then `python start.py`, then open a new Claude Code session.
- **If a start fails,** it now tells you why in 1 sentence on screen: the port was taken, the internet is not reachable and agent-flow is not saved on this computer yet, Node.js is missing or too old, or the package name in `config.json` is wrong. `python start.py --status` repeats that sentence later.
- **Stopping:** `python start.py --stop` says "Stopped agent-flow." or "agent-flow was not running. Nothing to stop." Check any time with `python start.py --status`.
- **If you move your vaults,** run `python install.py` again and give the new watch folder. The suggestion in square brackets is your OLD watch folder, so type the new one. If the server was running, the installer stops it and starts it again for the new folder.
- **Screen-sharing:** close the tab or pick a demo session first. Tab titles show the start of your prompts.
- **When you are not watching,** stop it with `python start.py --stop`. While it is stopped, no website can reach the page.

Checking, stopping, and stopping again

```
> python start.py --status
Running for C:\Users\<you>\Documents - http://127.0.0.1:3001
> python start.py --stop
Stopped agent-flow.
> python start.py --stop
agent-flow was not running. Nothing to stop.
```

ASKING WHETHER AGENT-FLOW IS RUNNING, STOPPING IT, THEN TRYING TO STOP IT A SECOND TIME, AS IT PRINTS.



A SUBAGENT AT WORK: EACH CARD IS 1 TOOL CALL IT MADE, HERE READING NOTES AND LISTING A FOLDER.

PIECE 1 • AGENT-FLOW

Fit it to your own AI system

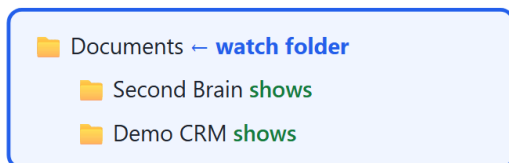
Change this download. It is yours now, and it is meant to be altered until it matches how you work. Ashley did exactly that with his own copy. He tried patching agent-flow to put every session in 1 tab, looked at the screen, saw 6 sessions stacked on the same spot, and threw the patch away. He worked out the 2 Windows fixes himself: start the server from the folder that sits above all your vaults, or it never hears your sessions; and delete the leftover registration files, or events stop arriving with no warning. He wrote the hidden start-at-logon file with usage tracking switched off. He added a count of hooks after his own settings file reached 12 copies of the same hook on every event. Every one of those fixes is in what you have downloaded.

Each idea below comes with a prompt you can paste into Claude Code, opened in the `outliers-ws-01-agent-flow` folder unless it says otherwise.

The watch folder must contain every vault you use

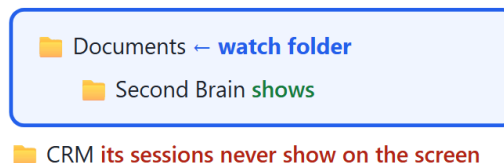
Works: both vaults inside Documents

📁 C:\Users\<you>



Fails: the CRM vault sits outside the watch folder

📁 C:\Users\<you>



Fix: make C:\Users\<you> the watch folder, because it contains both vaults.

IDEA 1 IN A PICTURE: THE WATCH FOLDER MUST CONTAIN EVERY VAULT WHOSE SESSIONS YOU WANT TO SEE.

1. **See your second brain AND your CRM in 1 place.** The server only hears sessions inside its watch folder. If your CRM lives at `C:\Users\<you>\CRM` and your second brain in `Documents`, the only folder containing both is your home folder.

```
Read config.json. Tell me whether my second brain vault and my CRM vault are both inside watch_folder. If not, run install.py again with --watch-folder set to the nearest folder that contains both, and show me the before and after.
```

2. **Watch your content system write a batch of posts.** Run whatever agents or scripts write your posts from a folder inside the watch folder, open agent-flow, and start a batch. You will see which steps call subagents and which run as plain code.

```
The agents or scripts that write my posts live at <path>. Check whether that folder is inside the watch_folder in config.json. If it is not, tell me the smallest change: move the folder, or widen the watch folder. Do not move anything without asking.
```

3. **Check an agent you are building really does what its instructions say.** Open the agent's file, list what it claims to call, run it with agent-flow open, and compare.

```
Read ~/.claude/agents/<agent-name>.md. List every subagent, tool and file it says it uses, as a checklist. I will run it now with agent-flow open and tell you what cards appeared. Then mark each checklist line as seen, not seen, or seen but not listed.
```

4. **Give every subagent a real name.** If your session starts a subagent that is not one of the named agents in your agents folder (the `.claude\agents` folder inside your home folder, where each agent has its own file), it shows as a hexagon with no name. Named agents make the picture readable.

```
Search my second brain's CLAUDE.md and my agent files for places that start a subagent with a generic type such as general-purpose. List each one and suggest which named agent from ~/.claude/agents/ should be used instead. Change nothing yet.
```

5. **Keep a permanent log.** agent-flow keeps no history once the server stops. Add a second, separate hook that appends each event to a dated file, then summarise a day into your second brain.

```
Add 1 NEW hook to my Claude Code settings.json on SubagentStart and SubagentStop that appends the event as 1 JSON line to ~/.claude/agent-events/YYYY-MM-DD.jsonl. Use a Python script run with pythonw.exe so no window opens. Take a dated backup of settings.json first, write it with a temp file and os.replace, and run python check_hooks.py afterwards to prove every event still has 1 copy of each command.
```

6. **Stop Node.js starting for nothing when the server is off.** If you only use agent-flow sometimes, every tool call still starts Node.js for nothing. Remove the hooks when you stop, and put them back when you start.

```
Write stop_all.py and start_all.py in this folder. stop_all.py runs start.py --stop and then removes the agent-flow hooks using common.remove_agent_flow, with a backup. start_all.py puts the hooks back with common.install_agent_flow and runs start.py. Add tests using a temporary home folder, like the ones in tests/, and run them with python -m pytest -q.
```

7. **Run it only when you want it.** Skip the file that starts agent-flow when you log in, and make a desktop shortcut instead. `--no-autostart` also removes that file if you already have it.

```
Run python install.py --no-autostart and check that outliers-agent-flow.vbs is no longer in my Startup folder. Then create a desktop shortcut called "Agent screen" that runs start.py with pythonw.exe (Windows) so no window appears, and a second one that runs start.py --stop.
```

8. **Capture footage for content.** Record the page while several agents are working at once: it gives you a short video of "the agents working" to use in a post.

```
Write capture.py: open http://127.0.0.1:3001 in headless Playwright at 1600 by 900, take a screenshot every 2 seconds until I press Ctrl+C, and save them numbered into a folder called captures with today's date. No visible browser window.
```

9. **Weekly hook check without a window.** Run the duplicate check once a week, with no window, and write the result into your second brain's inbox only when something is wrong. `check_hooks.py` ends with exit code 1 when it finds a duplicate and 2 when settings.json cannot be read (an exit code is the number a program hands back when it finishes; 0 means all well).

```
Create a Windows scheduled task that runs check_hooks.py once a week on Monday at 09:00 through pythonw.exe, hidden. If the exit code is 1 or 2, write a short note into <second brain>/Inbox/ named YYYY-MM-DD hook check.md with the output. Do not run it more often than weekly.
```

Every command and setting

outliers-ws-01-agent-flow: what is in the download

<code>install.py</code>	asks you 3 questions, finds your vaults, adds the hook, adds a file that starts agent-flow at logon
<code>start.py</code>	starts and stops agent-flow; checks settings.json first
<code>guard.py</code>	new: starts agent-flow, puts settings.json back if agent-flow rewrites it, refuses page requests from other websites
<code>check_hooks.py</code>	counts every hook on every event; add --fix to remove repeated copies
<code>cleanup.py</code>	deletes the small files a stopped agent-flow server left behind saying where it was listening, and ends a second server left watching the same folder
<code>common.py</code>	shared code the 5 files above use
<code>config.json</code>	made by install.py: your vault paths, watch folder, port
<code>logs/</code>	made at run time: start.log (the kit's notes), agent-flow.log (agent-flow's own output)
<code>tests/</code>	45 self-checks (run: python -m pytest -q); they use a made-up home folder, never your real one
<code>guide/</code>	this guide, as GUIDE.md and a PDF
<code>README.md</code>	a short summary and the useful commands
<code>WHAT-I-STOLE.md</code>	which of our files use code from agent-flow, and agent-flow's licence
<code>LICENSE</code>	the MIT licence: you may use and change our files freely

WHAT IS IN THE DOWNLOAD, AND WHAT EACH FILE DOES.

install.py (every option can be combined with the others)

OPTION	WHAT IT DOES
(none)	Asks you 3 questions, then installs.
<code>--yes</code>	Accepts every suggestion without asking.
<code>--second-brain <path></code> , <code>--crm <path></code>	Gives the vault paths instead of answering the questions.
<code>--watch-folder <path></code>	Sets the watch folder and skips the 3 questions. The installer warns you if a vault is not inside it.
<code>--port <number></code>	The page's port (default 3001). A running server is moved to the new port.
<code>--no-autostart</code>	Does not add the file that starts agent-flow at logon; also removes that file if you already have it.
<code>--start-now</code>	Starts the server at the end, so you can skip step 6.
<code>--uninstall</code>	Stops the server, removes the 9 hooks (backup first) and the file that starts agent-flow at logon.

OPTION	WHAT IT DOES
<code>--package</code> , <code>--node-path</code> , <code>--wait</code> , <code>--startup-dir</code>	Advanced and for testing. <code>--package</code> : which agent-flow version to run (default <code>agent-flow-app@0.9.1</code>). <code>--node-path</code> : which copy of Node.js the hook should use. <code>--wait</code> : how many seconds to wait for the first download (default 180). <code>--startup-dir</code> : a different Startup folder.

start.py

OPTION	WHAT IT DOES
(none)	Checks settings.json, deletes leftover registration files, starts agent-flow with no window, prints the address.
<code>--stop</code>	Stops the server this kit started.
<code>--status</code>	Says "Running" or "Not running", and if the last start was refused or failed, the 1-sentence reason. Exit code 0 when running, 1 when not.
<code>--port</code> , <code>--watch-folder</code> , <code>--package</code> , <code>--wait</code>	Override <code>config.json</code> for this run only.
<code>--foreground</code> , <code>--quiet</code>	Used by the files that start agent-flow when you log in. <code>--foreground</code> : stay running until agent-flow stops, instead of handing your terminal straight back, which the Mac login job needs. <code>--quiet</code> : print nothing at all, which the Windows file needs.

check_hooks.py: `--fix` keeps 1 copy of each repeated hook after a dated backup; `--settings <file>` checks a different settings file. Exit codes: 0 all well, 1 duplicates found, 2 the file cannot be read.

cleanup.py: `--dry-run` only reports what it would delete or end; `--quiet` prints nothing. It removes registration files for servers that have stopped, and when 2 servers are watching the same folder (what closing agent-flow by force leaves behind) it ends the older server and takes its file away.

Files the kit makes while it runs

- `config.json` : your 2 vault paths, the watch folder, the agent-flow version, the port and whether the start-at-logon file is on. `config.example.json` shows what the file looks like.
- `logs/start.log` : the kit's own notes, such as a refused start or a settings file put back.
- `logs/agent-flow.log` : agent-flow's own output. `start.py` reads its last 30 lines to work out the 1-sentence reason it prints when a start fails; open it yourself for the full text.
- `logs/agent-flow.pid` (PID stands for process ID, the number Windows gives the running program): that number and the server's start time, so `--stop` finds it and nothing else.
- Settings backups next to `settings.json` : `settings.json.bak-agent-flow-<date>` (before install), `.bak-agent-flow-uninstall-<date>` (before an uninstall), `.bak-hookdedupe-<date>` (before `--fix`) and `.bak-agent-flow-undo-<date>` (agent-flow's rewritten version, kept when `guard.py` put yours back).
- The file that starts agent-flow at logon: on Windows `outliers-agent-flow.vbs` in your Startup folder; on a Mac a login job, `~/Library/LaunchAgents/com.outliers.agent-flow.plist`, which runs at your next login (to start it now: `launchctl load` followed by that path).

Worth knowing

- If you set `CLAUDE_CONFIG_DIR` (a setting that tells Claude Code to keep its settings in a different folder), the installer uses that folder. agent-flow itself always uses `<home>/.claude`, so it will not find your session transcripts and you get a half-drawn page. See the "Half a page" row of "When it goes wrong".
- agent-flow also watches OpenAI Codex (OpenAI's coding assistant) unless the environment variable `AGENT_FLOW_RUNTIME=claude` is set. It does no harm if you do not use Codex.
- The kit's 45 automatic self-checks run with `python -m pytest -q`. They use a made-up home folder and a fake agent-flow, so your real settings are never touched. 4 more checks run against the real agent-flow if you first set the environment variable `AGENT_FLOW_REAL=1`; without it those 4 are skipped. Measured from a fresh copy on 2026-09-23: 45 passed, 4 skipped.

When it goes wrong

WHAT YOU SEE	WHY	FIX
The page says "WAITING FOR AGENT SESSION" forever	The session was already open when the server started, or it runs outside the watch folder.	Open a NEW session inside the watch folder. Check the folder with <code>python start.py --status</code> and <code>config.json</code> .
Half a page: tool call cards appear, but no subagent hexagons, the tab is titled <code>Session</code> and a number instead of your prompt, and the token count stays at 0	You have set <code>CLAUDE_CONFIG_DIR</code> to keep Claude Code's folder somewhere other than <code><home>/ .claude</code> . The hook still reaches agent-flow, but agent-flow reads session transcripts only from <code><home>/ .claude/projects/</code> , and yours are not there.	Start Claude Code without <code>CLAUDE_CONFIG_DIR</code> , or move the folder back to <code><home>/ .claude</code> . agent-flow cannot be pointed anywhere else; the folder is written into its code.
Still nothing after a new session	A leftover file from an old agent-flow server, pointing at a folder deeper inside your watch folder, is taking the events (Windows only).	<code>python cleanup.py</code> , then <code>python start.py --stop</code> and <code>python start.py</code> , then a new session.
"agent-flow was NOT started, to protect your Claude Code settings"	<code>settings.json</code> has a typing error, or an invisible marker at its very start that some editors add (a byte-order mark); agent-flow would have replaced the whole file.	Error with a line number: open <code>settings.json</code> at that line, fix it (a comma after the last item is the usual cause), save. Byte-order mark: run <code>python install.py</code> , which removes it after a backup. Then <code>python start.py</code> .
The page never loads after logging in	The settings check refused to start agent-flow at logon, or the start failed, and nothing was on screen to tell you.	<code>python start.py --status</code> prints the 1-sentence reason; fix as it says, then <code>python start.py</code> .
"agent-flow did not start", then a sentence	The start failed. The sentence names the cause: the port was taken, the internet could not be reached and agent-flow is not saved on this computer yet, Node.js is missing or too old, or the package name in <code>config.json</code> is wrong.	Do what the sentence says. For a taken port, run <code>python start.py</code> again. <code>logs/agent-flow.log</code> has the full text if you want it.
"Ended 1 agent-flow server(s) left behind when the last one was closed by force"	agent-flow was closed from Task Manager, or the computer shut down without <code>python start.py --stop</code> , so agent-flow outlived the program that started it.	Nothing to do. It has already been ended. Use <code>python start.py --stop</code> rather than Task Manager and it will not happen again.
Command windows flash on screen on every tool call	Copies of agent-flow's command have piled up in Claude Code's settings file. Before our scripts, agent-flow added 1 more copy every time it started on Windows.	<code>python check_hooks.py --fix</code> , then <code>python install.py</code> to rewrite the line with forward slashes.

WHAT YOU SEE	WHY	FIX
"Port 3001 is already in use"	Another program uses that port.	<code>python install.py --port 3002</code> , then <code>python start.py</code> , then open http://127.0.0.1:3002 .
The installer stops: Node.js not found	Node.js is not installed, or the terminal was opened before installing it.	Install it (see Before you start), open a NEW terminal, run again. Nothing was changed.
The installer stops: could not read settings.json	The file has a typing error in it. <code>check_hooks.py</code> shows the same fault with the line and column.	Open settings.json at that line, fix the error, run again. Your Claude Code settings were not changed.
<code>check_hooks.py</code> says settings.json is empty	The file is there but has nothing in it. agent-flow cannot read an empty file, so at its next start it would replace it with a file that has only its own 9 hooks in it. <code>start.py</code> refuses to start it for the same reason.	<code>python install.py</code> . It writes the 9 hooks into the file properly, after a backup.
A file called <code>settings.json.bak-agent-flow-undo-<date></code> appeared	agent-flow rewrote your settings after starting and <code>guard.py</code> (our program between agent-flow and your browser) put your version back. The file is agent-flow's version, kept for you to look at.	Nothing to do. <code>logs/start.log</code> records when it happened.
The token count and the cost on this page do not match the ones on another screen, such as FleetView	agent-flow's token and cost figures are estimates.	Treat them as a rough guide. They are not a bill.
A change you made to the screen "should work" but looks wrong	The change was checked by reading the numbers behind the picture instead of looking at the picture. On 2026-06-12 our attempt to show every session on 1 screen had the right data while the screen showed 6 hexagons stacked on 1 spot.	Take a screenshot and look at it before calling it done.

A settings.json that agent-flow would replace: the start is refused, --status repeats the reason, and check_hooks.py agrees. The last part of the picture is what an empty settings.json prints.

```
> python start.py
agent-flow was NOT started, to protect your Claude Code settings.
C:\Users\<you>\.claude\settings.json cannot be read (line 108, column 4: there is a comma after the
last item in a block). agent-flow would replace the whole file. Open it at that line, fix it and
save, then run: python check_hooks.py
> python start.py --status
Not running for C:\Users\<you>\Documents
Why the last start did not work: C:\Users\<you>\.claude\settings.json cannot be read (line 108,
column 4: there is a comma after the last item in a block). agent-flow would replace the whole file.
Open it at that line, fix it and save, then run: python check_hooks.py
> python check_hooks.py
Settings file: C:\Users\<you>\.claude\settings.json
RESULT: PROBLEM - C:\Users\<you>\.claude\settings.json cannot be read (line 108, column 4: there is a
comma after the last item in a block). agent-flow would replace the whole file. Open it at that line,
fix it and save.
start.py will not start agent-flow until this is put right, to stop agent-flow replacing the file.
Save your fix, then run this check again.
> python check_hooks.py (after saving an EMPTY settings.json)
Settings file: C:\Users\<you>\.claude\settings.json
RESULT: PROBLEM - C:\Users\<you>\.claude\settings.json is empty. agent-flow cannot read an empty
file, so at its next start it would replace it with a file that has only its own 9 hooks in it. Run:
python install.py
start.py will not start agent-flow until this is put right, to stop agent-flow replacing the file.
Save your fix, then run this check again.
```

A SETTINGS FILE WITH A COMMA AFTER ITS LAST ITEM: THE START IS REFUSED, `--STATUS` REPEATS THE REASON, AND `CHECK_HOOKS.PY` SAYS THE SAME WORDS. THE LAST PART OF THE PICTURE IS AN EMPTY SETTINGS.JSON FILE, WHICH `START.PY` AND `CHECK_HOOKS.PY` USED TO DESCRIBE DIFFERENTLY UNTIL 2026-09-23. MADE-UP HOME FOLDER, 2026-09-23.

A start that fails now names the cause in 1 sentence, and python start.py --status prints that same sentence again later (2026-09-23).

```
> python start.py
agent-flow did not start.
The port agent-flow needed was taken by another program a moment before it started. Run python
start.py again.
The full detail is in C:\Users\<you>\outliers-ws-01-agent-flow\logs\agent-flow.log
> python start.py --status
Not running for C:\Users\<you>\Documents
Why the last start did not work: The port agent-flow needed was taken by another program a
moment before it started. Run python start.py again.
```

A START THAT FAILED, SAYING WHY IN 1 SENTENCE INSTEAD OF POINTING AT A LOG. WE MADE THIS FAILURE HAPPEN ON PURPOSE ON 2026-09-23 BY STARTING ANOTHER PROGRAM ON PORT 3001 FIRST; THE WORDS ARE THE ONES OUR SCRIPTS REALLY PRINT.

2 agent-flow servers were watching the same folder after 1 was closed from Task Manager. cleanup.py counted them, ended the older one, and printed a line saying so (2026-09-23).

```
> python cleanup.py
Registration folder: C:\Users\<you>\.claude\agent-flow
Found 2 registration file(s): 1 running, 0 left over from stopped servers.
  KEEP    9b4417ec02aa5310-55116.json  port 53821  watching C:\Users\<you>\Documents
  ENDED   3f21a6c40b18e2d5-65936.json  (more than 1 agent-flow server was watching C:\Users\
<you>\Documents)
Ended 1 agent-flow server(s) left over after a server was closed by force. Each extra server
keeps a port open and draws the same events twice.
> python start.py
Ended 1 agent-flow server(s) left behind when the last one was closed by force.
agent-flow is running. Open http://127.0.0.1:3001
Start a NEW Claude Code session inside the watch folder to see it draw.
```

CLOSING AGENT-FLOW BY FORCE LEAVES A SECOND SERVER ON THE SAME FOLDER. `CLEANUP.PY` COUNTS HOW MANY AGENT-FLOW SERVERS ARE RUNNING, INSTEAD OF HOW MANY FOLDERS ARE BEING WATCHED, ENDS THE OLDER SERVER, AND THE NEXT START PRINTS A LINE SAYING IT DID. MADE-UP HOME FOLDER, 2026-09-23.

WARNING

do not run `npm run agent-flow-app` by hand while this kit is installed. You get 2 servers. Your sessions report to only 1 of them, the one watching the folder nearest to the session, and a copy started by hand runs without our settings check, so it can throw away settings.json. Use `python start.py` instead.

PIECE 1 • AGENT-FLOW

Download

The repository: <https://github.com/OUTLIERS-ai/outliers-ws-01-agent-flow>

```
git clone https://github.com/OUTLIERS-ai/outliers-ws-01-agent-flow; cd outliers-ws-01-agent-flow; python install.py
```

Then `python start.py` and open <http://127.0.0.1:3001>.

From download to a working screen in 4 steps



About 1 minute, plus the first download of agent-flow (up to 3 minutes on a slow connection).

THE 4 STEPS FROM DOWNLOAD TO A WORKING SCREEN.

agent-flow itself: github.com/patoles/agent-flow (Apache 2.0 licence). Our 5 scripts: MIT licence. Both are free licences: you may use and change the code.

DOWNLOAD THIS PIECE

<https://github.com/OUTLIERS-ai/outliers-ws-01-agent-flow>

```
git clone https://github.com/OUTLIERS-ai/outliers-ws-01-agent-flow
cd outliers-ws-01-agent-flow
python install.py
```